

# COMP1324 Algorithmics

## Dynamic Programming 2

Hikmat Farhat

# Longest Common Subsequence (Leetcode #1143)

- Consider two strings  $X$  and  $Y$ . Our goal is to find the longest sequence that is contained in both  $X$  and  $Y$ .
- For example:  $B$  **DB**  $D$  **C** and  $A$  **DBC** then the longest common sequence is  $DBC$
- Note that the characters in the sequence do not have to be necessarily consecutive
- This problem is **different** from the **common substring** problem where the characters in the common sequence are required to be consecutive.
- In this example the longest common substring is  $DB$

# Dynamic Programming Solution

- Consider two strings  $X = x_1x_2 \dots x_{n-1}x_n$  and  $Y = y_1y_2 \dots y_{m-1}y_m$ .
- We denote the prefix of length  $k$  of  $X$  by  $X_k = x_1 \dots x_k$  so that  $X = X_n$  and  $Y = Y_m$ .
- let  $LCS(X, Y)$  be the **length** of the longest common subsequence between  $X$  and  $Y$  then we can write  $LCS(X, Y) = LCS(X_n, Y_m)$ .
- If  $x_n = y_m$  then the common subsequence must include  $x_n$ . In this case

$$LCS(X_n, Y_m) = LCS(X_{n-1}, Y_{m-1}) + 1$$

- If  $x_n \neq y_m$  then  $x_n$  and  $y_m$  cannot be both in the solution

$$LCS(X_n, Y_m) = \max(LCS(X_n, Y_{m-1}), LCS(X_{n-1}, Y_m))$$

# Bottom up algorithm

First we note that if one substring has 0 elements the length of the common subsequence is 0. Therefore  $OPT[i][0] = 0$  and  $OPT[0][j] = 0$  for all  $i, j$

---

**Algorithm 1:** Longest Common Subsequence

---

**input:** Strings  $X$  and  $Y$

**Result:** Returns the optimal solution matrix

**function** LCS-DP( $X, Y$ )

```
    for  $i = 1$  to  $n$  do
        for  $j = 1$  to  $m$  do
            if  $X[i] = Y[j]$  then
                 $OPT[i][j] \leftarrow 1 + OPT[i-1][j-1]$ 
            else
                 $OPT[i][j] \leftarrow \max(OPT[i-1][j], OPT[i][j-1])$ 
    return  $OPT$ 
```

# Example

Consider the two strings  $X = ABCD$  and  $Y = AABDC$

| $X \backslash Y$ | "" | A | AA  | AAB | AABD | AABDC |
|------------------|----|---|-----|-----|------|-------|
| ""               | 0  | 0 | 0   | 0   | 0    | 0     |
| A                | 0  | 1 | ↖ 1 | 1   | 1    | 1     |
| AB               | 0  | 1 | 1   | ↖ 2 | 2    | 2     |
| ABC              | 0  | 1 | 1   | ↑ 2 | 2    | 3     |
| ABCD             | 0  | 1 | 1   | 2   | ↖ 3  | ← 3   |

Note that there are more than one optimal solution

# Finding the sequence

---

**Algorithm 2:** Longest Common Subsequence

---

**input:** Strings  $X$  and  $Y$

**Result:** Returns the size of the longest common subsequence

**function** LCS-SEQ( $X, Y$ )

```
     $L \leftarrow []$ 
    while  $i > 0 \wedge j > 0$  do
        if  $X[i] = Y[j]$  then
             $L.push\_front(X[i])$ 
             $i, j \leftarrow i - 1, j - 1$ 
        else
            if  $LCS[i - 1][j] > LCS[i][j - 1]$  then
                 $i \leftarrow i - 1$ 
            else
                 $j \leftarrow j - 1$ 
    return  $L$ 
```

# Knapsack

- Given a knapsack with capacity  $C$  and  $n$  items with weights  $w_1, \dots, w_n$  and values  $v_1, \dots, v_n$  the knapsack problem is given by:

$$\begin{aligned} &\text{maximize } \sum_i x_i v_i \\ &\text{subject to } \sum_i x_i w_i \leq C \end{aligned}$$

- where  $x_i$  is the number of instances of item  $i$  included in the solution

# Variants

- There are many variants of the knapsack problem
- The **bounded knapsack**.  $x_i \in \{0, 1, \dots, c\}$
- The **unbounded knapsack**.  $x_i \in \mathbb{N}$
- The **0-1 knapsack**.  $x_i \in \{0, 1\}$
- The **fractional knapsack**.  $x_i \in \mathbb{R}, x_i \geq 0$ . This is an approximation algorithm. Also its solution is an upper bound for the optimal solution of the 0-1 knapsack problem.

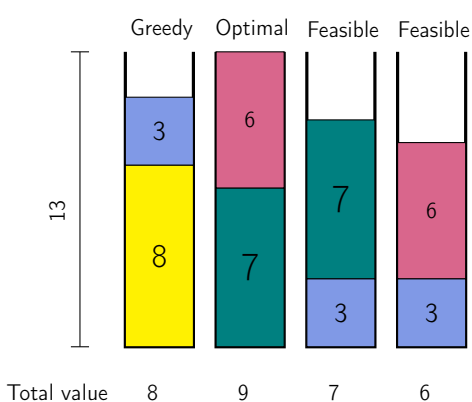
# 0-1 Knapsack Example

Consider the following instance of the 0-1 knapsack problem where the knapsack capacity is  $C = 13$ .

|           |           |                          |
|-----------|-----------|--------------------------|
| $v_1 = 2$ | $w_1 = 3$ | $\frac{v_1}{w_1} = 0.66$ |
| $v_2 = 4$ | $w_2 = 6$ | $\frac{v_2}{w_2} = 0.66$ |
| $v_3 = 5$ | $w_3 = 7$ | $\frac{v_3}{w_3} = 0.71$ |
| $v_4 = 6$ | $w_4 = 8$ | $\frac{v_4}{w_4} = 0.75$ |

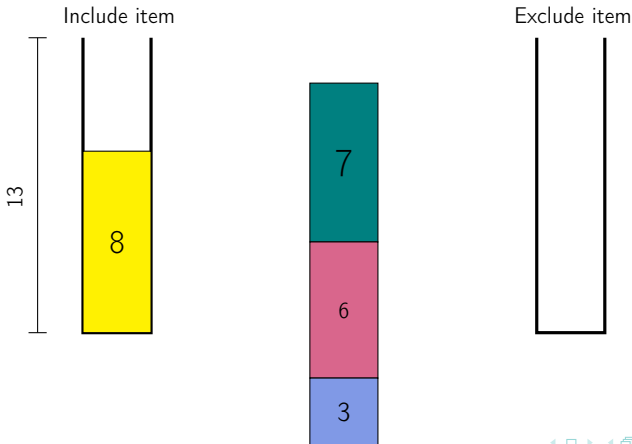
- Note that a greedy approach would choose item 4 then 1 for a total value of 8
- The optimal is 9 by choosing items 3 and 2.
- The greedy solution is optimal for the **fractional** knapsack
- It can be used as an upper bound.

# Example



# Optimal substructure

- To relate the solution of the problem with 4 items to the solution of smaller subproblems.
- Either include the last item (weight 8) in the solution or not.



# Dynamic Programming solution for 0-1 knapsack

Let  $S_n$  be the optimal solution when the problem contains  $n$  items with knapsack capacity  $C$ . Consider item  $i_n$ . There are two possibilities:

# Dynamic Programming solution for 0-1 knapsack

Let  $S_n$  be the optimal solution when the problem contains  $n$  items with knapsack capacity  $C$ . Consider item  $i_n$ . There are two possibilities:

- 1 Either  $i_n \notin S_n$ .

# Dynamic Programming solution for 0-1 knapsack

Let  $S_n$  be the optimal solution when the problem contains  $n$  items with knapsack capacity  $C$ . Consider item  $i_n$ . There are two possibilities:

- ① Either  $i_n \notin S_n$ .
  - That leaves us with  $n - 1$  remaining items and a knapsack of capacity  $C$ .

# Dynamic Programming solution for 0-1 knapsack

Let  $S_n$  be the optimal solution when the problem contains  $n$  items with knapsack capacity  $C$ . Consider item  $i_n$ . There are two possibilities:

- ① Either  $i_n \notin S_n$ .
  - That leaves us with  $n - 1$  remaining items and a knapsack of capacity  $C$ .
- ② Or  $i_n \in S_n$ .

# Dynamic Programming solution for 0-1 knapsack

Let  $S_n$  be the optimal solution when the problem contains  $n$  items with knapsack capacity  $C$ . Consider item  $i_n$ . There are two possibilities:

- ① Either  $i_n \notin S_n$ .
  - That leaves us with  $n - 1$  remaining items and a knapsack of capacity  $C$ .
- ② Or  $i_n \in S_n$ .
  - That leaves us with  $n - 1$  remaining. But since we have decided to include item  $i_n$  the remaining capacity is  $C - w_n$ .

# Dynamic Programming solution for 0-1 knapsack

Let  $S_n$  be the optimal solution when the problem contains  $n$  items with knapsack capacity  $C$ . Consider item  $i_n$ . There are two possibilities:

- ① Either  $i_n \notin S_n$ .
  - That leaves us with  $n - 1$  remaining items and a knapsack of capacity  $C$ .
- ② Or  $i_n \in S_n$ .
  - That leaves us with  $n - 1$  remaining. But since we have decided to include item  $i_n$  the remaining capacity is  $C - w_n$ .

Because we don't know a priori if  $i_n \in S_n$  we compute both values and take the largest.

# Recurrence

- Because we don't know a priori if  $i_n \in S_n$  we compute both values and take the largest.

$$S(n, C) = \max \begin{cases} v_n + S(n-1, C - w_n) \\ S(n-1, C) \end{cases}$$

- Before writing a bottom up solution we need to take care of the boundary conditions.
- Anytime the capacity of the knapsack  $C = 0$  the solution is just 0.
- Also we need to take into account the case when  $w_n > C$ . In this case  $S(n, C) = S(n-1, C)$

---

**Algorithm 3:** Bottom up solution for 0-1 Knapsack

---

**input:** arrays of values  $v$ , weights  $w$  of size  $n + 1$  where  $v[0]$  and  $w[0]$  are unused. Capacity  $C$

**Result:** Returns matrix  $OPT$  of size  $(n + 1, m + 1)$ , the optimal values

```
function Knapsack( $w, v, C$ )  
    for  $i = 1$  to  $n$  do  
        |  $OPT[i][0] = 0$   
    for  $i = 1$  to  $C$  do  
        |  $OPT[0][i] = 0$   
    for  $j = 1$  to  $C$  do  
        | for  $i = 1$  to  $n$  do  
        |     | if  $j \geq w[i]$  then  
        |     |     |  $OPT[i][j] = \max(OPT[i - 1][j], v[i] + OPT[i - 1][j - w[i]])$   
        |     | else  
        |     |     |  $OPT[i][j] = OPT[i - 1][j]$   
    return  $OPT$ 
```

## Solution matrix for example

| $n \backslash w$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|------------------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  |
| 1                | 0 | 0 | 0 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2  | 2  | 2  | 2  |
| 2                | 0 | 0 | 0 | 2 | 2 | 2 | ④ | 4 | 4 | 6 | 6  | 6  | 6  | 6  |
| 3                | 0 | 0 | 0 | 2 | 2 | 2 | 4 | 5 | 5 | 6 | 7  | 7  | 7  | ⑨  |
| 4                | 0 | 0 | 0 | 2 | 2 | 2 | 4 | 5 | 6 | 6 | 7  | 8  | 8  | 9  |

# Finding the solution

---

**Algorithm 4:** Bottom up solution for 0-1 Knapsack

---

**input:** optimal solution matrix  $OPT$  and array of weights  $w$  with  $w[0]$  unused. Knapsack capacity  $C$

**Result:** Returns list of items making up the optimal solution

**function** Knapsack( $OPT, w, C, n$ )

$w \leftarrow C$

$L \leftarrow [ ]$

$i \leftarrow n$

**while**  $i > 0 \wedge w > 0$  **do**

**if**  $OPT[i][w] > OPT[i - 1][w]$  **then**

$L.push\_front(i)$

$w \leftarrow w - w[i]$

$i \leftarrow i - 1$

**return**  $L$

---

## Optional: Longest path

- Given a directed acyclic graph (DAG)  $G = (V, E)$
- Use DP to compute the longest path in  $G$
- Let  $dist[u]$  the longest distance from source  $s \in V$  to  $u$ .
- We can write

$$dist[v] = \max_{(u,v) \in E} (dist[u] + w(u, v))$$

- To make sure  $dist[u]$  is computed before  $dist[v]$  we run topological sort on  $G$  first.
- Both the above algorithm and topological sort are  $O(|V| + |E|)$  as opposed  $O(|V| \cdot |E|)$  for Bellman-Ford (see next tutorial)

# Optional: Longest path in DAG

---

**Algorithm 5:** Longest Path: DP Computation

---

**Input** : DAG  $G$ , Topological Sort  $L$ , Source  $s$

**Output:** Global arrays  $dist$  and  $prev$

**foreach**  $v \in V$  **do**

$dist[v] \leftarrow -\infty$ ,  $prev[v] \leftarrow \text{null}$

$dist[s] \leftarrow 0$

**foreach**  $u \in L$  **do**

**if**  $dist[u] \neq -\infty$  **then**

**foreach**  $v \in adj[u]$  **do**

**if**  $dist[v] < dist[u] + w(u, v)$  **then**

$dist[v] \leftarrow dist[u] + w(u, v)$

$prev[v] \leftarrow u$

---

# Optional

---

## Algorithm 6: Longest Path: Reconstruction

---

**Input** : Target  $t$ , Array  $prev$  from Part 1

**Output**: Reconstructed Path  $P$

$path \leftarrow []$

$curr \leftarrow t$

**while**  $curr \neq null$  **do**

    Append  $curr$  to  $path$

$curr \leftarrow prev[curr]$

$P \leftarrow reverse(path)$

**return**  $P$

---

# References

- The longest common subsequence is covered in section 14.4 of your [textbook](#)
- A Python implementation of the longest common subsequence problem can be found [here](#)
- A Python implementation of the knapsack problem can be found [here](#)